Diseño modular y arquitectura

STOCKFLOW

Sistema de Ventas de Aparatos Electrónicos de Escritorio

Ciclo: 1 | **Etapa:** DIS | **Artefacto:** Documento de Arquitectura de Software y Diseño Modular (DAS)

Control de Versiones

Parámetro	Detalle
Nombre del documento	STKD_DAS_V0_1_191026.pdf
Fecha	19-Oct-2026
Razón del cambio	Definición de la arquitectura de software, estilo arquitectónico en capas (MVC/DAO), diseño modular, diagrama de componentes y paquetes Java para el sistema de escritorio.
Autor	DAOJ (Diego Armando Ojeda Jiménez - Arquitectura y decisiones técnicas)
Revisión / Apoyo	VMTS (Víctor Manuel Tequiliquihua de los Santos), KTCG (Kevin Tomás Caballero Guzmán)
Versión	0.1
Métricas de Tiempo	Inicio: 17/10/2026

1. Introducción

En este documento presento la especificación arquitectónica y el diseño modular para el Sistema de Ventas de Aparatos Electrónicos de Escritorio (**StockFlow**). Este artefacto toma como entrada los requerimientos definidos en el SRS (STKD_SRS_V0_2_220926) y las maquetaciones visuales establecidas en el documento de pantallas de mostrador (STKD_MPM_V0_2_270926), estableciendo la estructura técnica necesaria para la fase de construcción en Java Swing y la persistencia local.

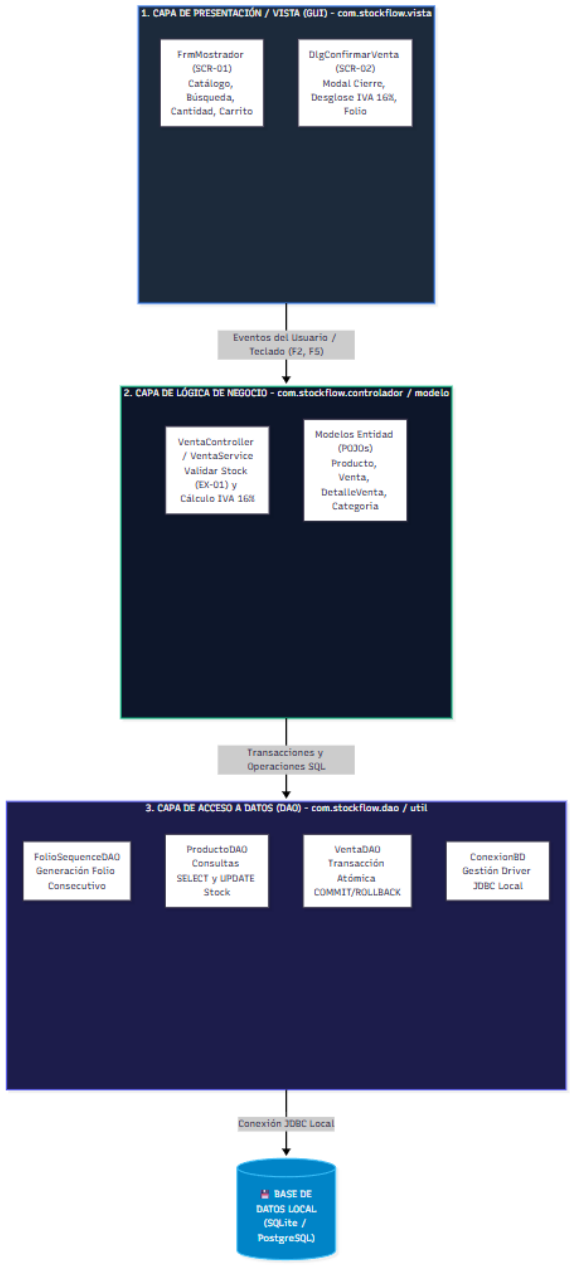
1.1 Objetivos Arquitectónicos

Diseño modular y arquitectura

- **Separación de Responsabilidades:** Aplicar el patrón arquitectónico de tres capas (Presentación, Lógica de Negocio y Acceso a Datos / DAO) en cumplimiento con el requisito de mantenibilidad RNF-08.
- **Persistencia Atómica Local:** Garantizar la integridad transaccional (RNF-03) para que el registro de la venta y el descuento de inventario se ejecuten como una sola operación indivisible en la base de datos local (RNF-04).
- **Bajo Acoplamiento y Alta Cohesión:** Diseñar módulos independientes que permitan al equipo de desarrollo (AESA en la lógica funcional y VMTS en la base de datos) implementar e integrar componentes sin interferir en el diseño visual de las pantallas.

2. Vista General de la Arquitectura (Patrón Tres Capas)

La arquitectura del sistema **StockFlow** sigue un estilo en **3 Capas (Layered Architecture)** implementado mediante el patrón **MVC (Modelo-Vista-Controlador)** desacoplado con patrones **DAO (Data Access Object)**:



Diseño modular y arquitectura

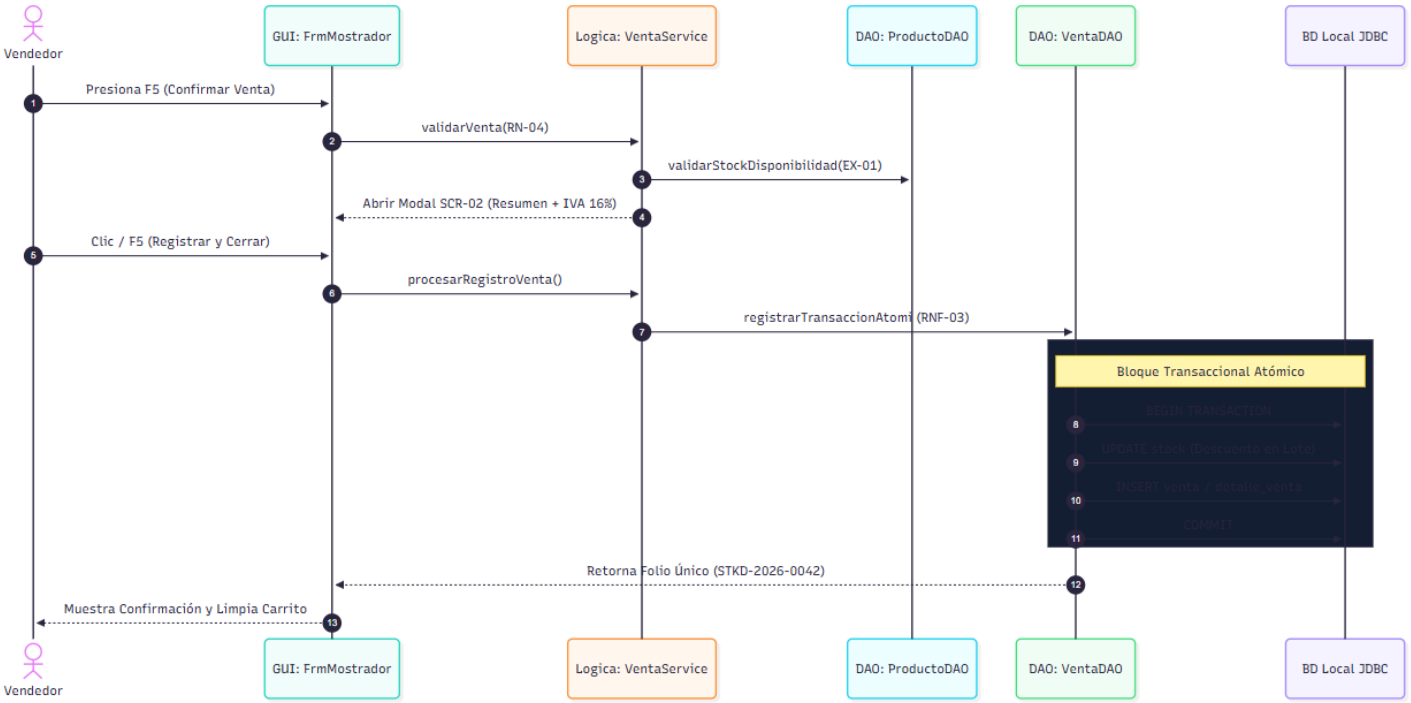
3. Diseño Modular y Paquetes Java

Para organizar el código fuente en la etapa de desarrollo (EDT 1.3), dividí el sistema en los siguientes módulos principales dentro del espacio de nombres com.stockflow:

Módulo / Paquete	Clases Principales	Propósito y Responsabilidad	Responsable EDT (Rol) PDF+ 1
com.stockflow.vista	FrmMostrador, DlgConfirmarVenta, PanelTablaCatalogo	Contiene los componentes gráficos en Java Swing (JTable, JTextField, JButton). Muestra la información de mostrador y captura los eventos de teclado (F2, F5, ESC).	DAOJ (Arquitecto / UI)
com.stockflow.modelo	Producto, Venta, DetalleVenta, Categoria	Clases POJO (Plain Old Java Objects) que representan los datos de negocio con sus atributos, métodos *getters/setters* y operaciones de encapsulamiento.	AESA (Desarrollo Funcional)
com.stockflow.controlador	VentaService, CatalogoService, CalculadoraImpuestos	Contiene las Reglas de Negocio: cálculo de subtotales, IVA (16%), validación de stock físico disponible (RN-02, EX-01) e impedimento de ventas vacías (RN-04).	AESA (Desarrollo Funcional)
com.stockflow.dao	ProductoDAO, VentaDAO, FolioGeneratorDAO	Interfaz y clases de acceso a datos SQL (JDBC). Ejecuta consultas SELECT, sentencias INSERT y actualizaciones de stock UPDATE dentro de un bloque transaccional atómico (commit/rollback).	VMTS (Persistencia / BD)
com.stockflow.util	ConexionBD, FormateadorMoneda, GestorAtajosTeclado	Clases utilitarias globales: conexión a base de datos local (RNF-04), formateo de precios en MXN (DecimalFormat) y manejo centralizado de atajos por teclado.	VMTS / DAOJ

4. Diagrama de Secuencia: Flujo de Cierre de Venta y Folio

El siguiente diagrama detalla la interacción interna entre componentes cuando el vendedor presiona la tecla **F5** para confirmar una compra:



5. Decisiones Técnicas y Mecanismos de Calidad (RNF)

1. **Gestión Atómica de Transacciones (RNF-03 / RN-06):** Para evitar inconsistencias (como descontar stock pero fallar al guardar la venta), el método VentaDAO.registrarVenta() desactiva el *AutoCommit* (connection.setAutoCommit(false)). Se ejecutan los INSERT en la tabla de ventas y el UPDATE en existencias dentro de un único bloque try-catch. Si ocurre un error, se ejecuta inmediatamente connection.rollback() (EX-03).

2. **Generación de Folios Únicos Consecutivos (RF-03.3 / RN-05):** El sistema genera un folio con el formato STKD-YYYY-XXXX (ej. STKD-2026-0042). Este folio se calcula mediante una secuencia en la base de datos local únicamente cuando la transacción de cobro ha sido confirmada en la etapa 11 del flujo de usuario.

3. **Manejo Desacoplado de Eventos en Interfaz:** Los componentes de la interfaz no contienen código SQL ni reglas de negocio. Escuchan los eventos del mouse y atajos de teclado (F2, F3, F5, ESC) y delegan la lógica al VentaService.

4. **Independencia de Red y Persistencia Local (RNF-04 / RNF-05):** La base de datos se ejecuta en el almacenamiento SSD del equipo local con soporte para respaldos manuales en archivo (.sql/.db) (RNF-06).

Diseño modular y arquitectura

6. Trazabilidad de Arquitectura vs. Requisitos del SRS

Requisito / Calidad	**Componente Arquitectónico**	**Mecanismo de Implementación**
RF-01.1 / RF-01.2 (Consulta y Búsqueda)	CatalogoService / ProductoDAO	Consultas filtradas SQL con LIKE por código/nombre y cláusula WHERE categoria = ?.
RF-02.1 / EX-01 (Validar Stock)	VentaService.validarStock()	Método en Java que compara spnCantidad contra producto.getStockDisponible() enviando la alerta visual.
RF-02.2 (Cálculo de Totales e IVA)	CalculadoraImpuestos	Algoritmo centralizado que desglosa el **Subtotal** y el **IVA (16% incl.)** tras cada evento de cambio en la lista.
RF-03.2 / RNF-03 (Transacción Atómica)	VentaDAO.registrarVenta()	Manejo de transacciones JDBC mediante commit() y rollback() en base de datos local.
RF-03.3 (Folio Único)	FolioGeneratorDAO	Secuencia atómica incremental en BD asignada tras el cierre del registro de compra.
RNF-08 (Mantenibilidad / Arquitectura)	Paquetes vista, modelo, controlador, dao	Arquitectura en 3 Capas MVC/DAO que separa la UI de la base de datos.

7. Conclusión

El presente Documento de Arquitectura de Software (**DAS**) establece la estructura técnica, modular y transaccional para el proyecto **StockFlow**. Al definir con claridad la división por capas, la convención de paquetes en Java y los mecanismos atómicos para el manejo de inventario y folios, se garantiza que la siguiente fase del cronograma (EDT 1.3: *Desarrollo del Sistema*) se ejecute de manera organizada, ágil y sin ambigüedades técnicas entre los integrantes del equipo.